# METACOGNITION ALGORITHM AND ITS COGNITIVE PROCESSES

Analyzing 50 random ARC-AGI1 training puzzles (see [20] in Doc), it seems that we can arrive at a sufficiently stable version of the general HiS-ARC(**H**uman **I**nspired **IS** for **A**bstract **R**easoning **C**orpora) algorithm. We can think this algorithm as the *metacognition* of HiS-ARC(see [2, 8]) written in the language of IST. Another way to describe this process of discovering the metacognition algorithm is: "Taking inspiration from the human way to solve the puzzles, we formalize this behavior using the language of IST, and we then implement it in Python". For this motivation, and to highlight the potential connections with neuroscience and cognitive psychology, we tried to name the different parts of this algorithm using a terminology inherited from these two sciences (as similarly done by, e.g., [19]). Even if the same possibility to realize this human-inspired formalization process is, per se, meaningful, it does not imply that this formalization *faithfully* corresponds to human cognitive processes, which is an open research question.

Note that ARC-AGI 2 seems to be only an extension of ARC-AGI 1: up to 9 exclusive ARC-AGI 1 puzzles, all the other 397 puzzles of ARC-AGI 1 are included in ARC-AGI 2. See `ARC-AGI 1_and_2.py`. After a random partial checking, it seems that ARC-AGI 2 puzzles are not conceptually different from ARC-AGI 1.

## 1. Core-knowledge

*Core-knowledge* seems the correct name for the set of all our Python codes aiming to recognize interacting entities, their relations/interactions, their states, and actions[1] that transform these entities and their relations. It corresponds to the innate cognitive abilities of newborns, starting from which they are able to construct a more advanced knowledge, see [9, 13].

### 1.1. **Recognition of entities.**

1.1.1. *Entities as IS.* It is important to understand that frequently humans are able to recognize some entities as a whole, even if they are themselves made of interacting sub-entities. A typical example is a filled square made of cells of different colors: sometimes it interacts as a square, but sometimes we have to consider that it is made of differently colored cells (and maybe their interactions have to be preserved from $\mathcal{I}_k$ to $\mathcal{S}_k$). We can formalize this behavior by representing this type of entities as IS themselves and made of these sub-entities (frequently interacting with a relation of the type "touching"). We can call this type of objects *complex entities* (we can also think to implement each entity as a, possibly trivial, complex one). These sub-entities also enter as entities of $\mathcal{I}_k$ or $\mathcal{S}_k$. For example, we can see a rectangle as an IS made of touching L-forms (or even, only 4 cells working as vertices), with boundaries rows and columns (having a certain thickness), interior cells, border cells, missing cells (each one with its own color) in case the rectangle is incomplete, etc. Similarly, we can see an L-form as an IS made of two arms touching at an angle cell, each one of these entities having its own color, thickness, etc.

It is also useful to define a fuzzy-logic based indicator which measure "how much this complex entity is near to a complete regular entity". In case of rectangle, this dopamine-like indicator, see Sec. 3, increases e.g. with "number of L-form instead of single cells as vertices", "fuzzy measure of completeness of sides", "unique color for interior cells", etc.

Among the state of these IS-entities, we always have to consider the "number of interacting sub-entities it is made of".

Important examples going in this direction are:

- L-, T-, I- and cross forms.
- Linear patterns: they are made of a few number of (not necessarily equal) figures repeated and touching, possibly with a separator. If these figures have all the same height, we have an horizontal linear pattern, otherwise we have a vertical one. Starting with the "first" or "last" figure (i.e. those touching only one other entity and not two of them), linear patterns define two linear order, the former being the opposite of the latter. This order is frequently used in puzzles.
- Tiling: if we see them as IS, we can also use cognitive and analogical mappings (see below Sec. 5) to decide whether they are tiling or not, or if they are complete tiling or not. The incomplete parts (e.g. where

[1] *Action* is a more correct name, as used e.g. in [1, 6], with respect to the old one *transformation*, which is only related to static geometrical transformations as used in ARC-AGI 1, 2.

the internal cognitive mappings do not preserve interactions) can then be used for an action of the type "complete an incomplete tiling".
- Symmetric figures or incomplete symmetric figures.
- Closed lines (where the borders may have a thickness, i.e. they are made by more that 1 cell of width, or different colors).

1.1.2. *Sets of cells as entity.* A geometric entity such as "set of cells" clearly always occur in each training $\mathcal{I}_k$ and $\mathcal{S}_k$, but they are usually not recognized by humans in this abstract way. Indeed, human do not think at them as possible interacting entity if we are able to recognize for them other geometric entities. For this reason, it is better to construct a core-knowledge which is able to search for "sets of cells having a certain property", such as:

1. set of all the cells that are interior of ...
2. set of all the cells that are exterior of ...
3. Set of all the cells that are above/right/left of ...
4. Set of all the cells that are contiguous ... (*weakly contiguous* if some of these cells touch only at a vertex, *contiguous* if these cells always touch on sides)
5. Set of all the cells shared by ...
6. Set of all the cells that are of the same color and do not correspond to any other recognizable entity.

1.1.3. *A posteriori recognition of entities.* As we will see below, some entities are recognized only a posteriori: this typically includes entities that are clear in $\mathcal{S}_k$ but do not have a corresponding analogous entity in $\mathcal{I}_k$ (i.e. they do not have a counter-image along an analogical or cognitive mapping, see Sec. 5 and Sec. 6), or entities emerging from missing parameters in transformations/actions. Only in this way, HiS-ARC recognizes *black* entities like diagonals, cells, squares, etc. See e.g. the puzzle `3906de3d`, where HiS-ARCcan start with the hypothesis "set of contiguous blue cells" and "red columns" but only after constructing a first version of vertical and horizontal analogical mappings (see Sec. 5) it can recognize "black columns" touching the "set of contiguous blue cells".
*Remark* 1.

1. In order to construct forgetful CEF, in every training IS $\mathcal{I}_k$, $\mathcal{S}_k$ we always have to consider "the empty interacting entity" $\emptyset$ and the interaction $\emptyset \xrightarrow{\emptyset} \emptyset$. An entity such that $U(e) = \emptyset$ is forgotten.
2. In each training IS, the bar (linear pattern?) containing the 10 colors $0, \dots 9$ is always an interacting entity. The transformation of HiS-ARC can in fact include "color the object $e$ with the color of $e'$" or "color the object $e$ with the color $k \in \{0, \dots, 9\}$". The $k$-th color could also not occur in $\mathcal{I}_k$ (but the chosen color $k$ is frequently, but not always, the same in all the training IS).
3. From the human point of view, a rectangle has a height, width of at least 2 rows resp. 2 columns; otherwise, it is a row or a column.

1.2. **Recognition of interactions.** See the issue "Lists of geometrical entities #2" on GitHub for a sufficiently complete list of interactions used in solving these puzzles.

1.2.1. *Sets = only trivial (identity) interactions.* Sometimes, see e.g. `e98196ab`, HiS-ARC can arrive to hypothesize (see Sec. 9) that there are no interactions in $\mathcal{I}_k$. In terms of category theory, this means that $\mathcal{I}_k$ is only a set; in this case, the only trivial interactions are the identity $e \xrightarrow{\text{is equal to}} e$ that are trivially preserved (therefore, we do not have to implement them in the code). In these cases, HiS-ARC can produce an explanation of the type: ``I recognized that there are only trivial interactions (equality) in $\mathcal{I}_k$''.

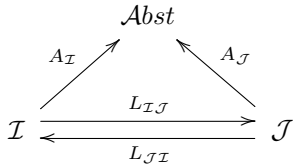
1.2.2. *A posteriori recognition of interactions.* As for entities, an a posteriori recognition of interactions can also occur using analogical mappings or in exploring some hypotheses, see below Sec. 5, Sec. 9 and Sec. 10.

For example, sometimes HiS-ARC recognizes (e.g. under certain hypotheses, see again Sec. 9) that the considered cognitive mappings (see Sec. 6) do not preserve some hypothesized interactions: this open a new branch hypotheses of deleting these interactions and/or to consider another hypothesis in HiS-ARC metacognition. See e.g. `beb8660c` where the (analogically weak) "touching" relations are deleted in favor of "has a lower number of patterns".
*Remark* 2.

1. Implicit interactions HiS-ARC always considers are "touching the border of the background grid" or "touching the vertices of the background grid".

## 2. Abstractions

From the mathematical point of view, an *abstraction* is a situation of the form




where $\mathcal{I}$, $\mathcal{J}$, $\mathcal{A}bst$ are IS and all the arrows are CEF; then the interacting entities $e \in \mathcal{I}$ and $\bar{e} \in \mathcal{J}$ correspond to the same abstract concept $\alpha \in \mathcal{A}bst$ if $L_{\mathcal{I}\mathcal{J}}(e) = \bar{e}$, $L_{\mathcal{J}\mathcal{I}}(\bar{e}) = e$ and $A_{\mathcal{I}}(e) = \alpha = A_j(\bar{e})$. Example: the entities $e$ and $\bar{e}$ are two rectangles (maybe of different dimension or color), $\alpha$ is, trivially, the string ``rectangles'' and the CEF between $\mathcal{I}$, $\mathcal{J}$ takes vertex and sides of the rectangles preserving their touching interactions; note that this CE preservation is the non trivial part of the definition because it includes human inspired behavior: e.g. "the entities $e$ and $\bar{e}$ are both rectangles because this L-form corresponds to this, and this L-form to this other one... and this side corresponds to this one, etc. All these correspondences preserve the correct touching interactions" (think at an "open" rectangle where one side does not close correctly or to a situation where one of the L-form is not correctly oriented: human do not think they are rectangles anymore).

There are two ways of constructing the IS of abstract notions $\mathcal{A}bst$:

1. In the first intrinsic way, we define the horizontal CEF by mimicking human behavior; the idea is that the property
$$\exists L_{\mathcal{I}\mathcal{J}}, L_{\mathcal{J}\mathcal{I}} \text{ CEF}: \; L_{\mathcal{I}\mathcal{J}}(e) = \bar{e}, \; L_{\mathcal{J}\mathcal{I}}(\bar{e}) = e$$
defines an equivalence relation $e \sim \bar{e}$; we can finally construct (automatically?) the IS $\mathcal{A}bst$ as the space of these equivalence classes (e.g. representing rectangles, triangles, straight lines, etc.) and abstracting from $\mathcal{I}$, $\mathcal{J}$ the interactions of these entities (e.g. move, dilate, rotate, etc.). We will not implement this intrinsic way in HiS-ARC, even if it seems a very general method more corresponding to human behavior.
2. In the second way, we directly define the IS $\mathcal{A}bst$ and its interactions by mimicking what occurs in $\mathcal{I}$, $\mathcal{J}$, inspired by human knowledge and considering the problem we have to solve. In HiS-ARC we use this method: in the state of each interacting entity and interactions that HiS-ARC recognizes in the input training $\mathcal{I}_k$, $\mathcal{S}_k$, it also includes the name of its abstract concept: `L-form`, `rectangle`, `diagonal`, `X is contained in Y`, `X touches Y on Z`, `X shares Y with Z`, etc. This state properties are used to construct entities and interactions of the IS $\mathcal{A}bst$, and the abstraction CEF (cognitive mappings) $A_{\mathcal{I}_k}$ and $A_{\mathcal{S}_k}$.

The general rule is that: *For everything we introduce in the code, there must be a corresponding abstract description in $\mathcal{A}bst$ and, possibly, the related interactions.*

### 2.1. Higher level abstract entities of $\mathcal{A}bst$.

In HiS-ARC, we also have to consider interactions of the form e.g.
$$i : \text{rectangles} \xrightarrow{r} \text{closed lines}$$
$$j : \text{triangles} \xrightarrow{s} \text{closed lines}$$
$$k : \text{squares} \xrightarrow{u} \text{rectangles}$$
$$h : \text{L-shapes} \xrightarrow{v} \text{L-shapes},$$
where the propagators have the following properties:

1. $\text{ac}_i^r(t) = 1$ means that at time $t$ (computation time) we have found a `'rectangle'` in at least one training $\mathcal{I}_k$ but it occurs also in the others $\mathcal{I}_j$ only if we think at it as a more abstract ``closed line'' (e.g. because in $\mathcal{I}_j$ there is a `'triangle'`). Similarly for $\mathcal{S}_k$. For example, if a `'rectangle'` occurs in $\mathcal{I}_1$ and $\mathcal{I}_2$, and a `'square'` in $\mathcal{I}_3$, we have an occurrence of `'rectangle'`. If we have `'rectangle'` in $\mathcal{I}_1$, a `'triangle'` in $\mathcal{I}_2$ and a `'circle'` in $\mathcal{I}_3$, then we have an occurrence of `'closed lines'`. This would correspond to a certain application of the GEP in HiS-ARC. However, since it is a strict rule, it would correspond to maximum cost $C_y^j(e) = 1$ where $C_y^j(e) \in \{0, 1\}$, or zero probability $Q_\gamma(i) = 0$. Up to now, we can therefore not use the GEP but simply apply this strict rule.
2. $\gamma_i(t)$ is a counting good following *synaptic plasticity*, i.e. "how many times we used this abstraction so far", see Sec. 4 below for the corresponding implementation.
3. An interaction of the type $h : g \xrightarrow{v} g$, where $g$ is an abstract geometrical objects, simply means that $g$ occurs in some $\mathcal{I}_k$. In the state $x_v(t)$, we include some kind of link to the code used to recognize the entity $g$: if all the names of functions used to recognize an entity are held in a dictionary `core_knowledge_entities` with

entries `('description_of_funct1': func1_name)`, then `core_knowledge['description_of_func1']()` works exactly as `funct1_name()`. In this way, we can say that "our code is interacting as interactions of HiS-ARC".

2.2. **Abstract interactions in $\mathcal{A}bst$.** Also geometric relations (interactions of $\mathcal{I}_k$, $\mathcal{S}_k$) and actions/transformations have to be organized in a hierarchical way using a similar abstraction, like geometric entities as above. These interactions generalize the list of geometric relations. For example:

$$j : \text{cells} \xrightarrow{r} \text{rectangle}$$

$$k : \text{rectangles}, \text{square} \xrightarrow{s} \text{lines},$$

with `j.pr.type = ''contains''`, `k.pr.type = ''share a line''`, and $\gamma_j$, $\gamma_k$ are "counting goods" as above. In the state of these propagators we include some kind of link to the code that recognize this relation (e.g. the dictionary as above).

Therefore, each actions has a related abstract description as an interaction of the form

$$\text{abstr } a_1, \dots, \text{abstr } a_n \xrightarrow{\text{type of action}} \text{abstr } p,$$

where "abstr $a_j$" represents an abstract interacting entity of HiS-ARC (e.g. a "diagonal" or "L-form", etc.) and "type of action" is a string describing the action itself. Among the agents of this description, we also include the parameter this action needs to run. In the state of the propagator (i.e. the state of the action/interaction itself), we insert the name of the functions in the `dict` containing all actions.

2.3. **Abstract actions/transformations.** These interactions corresponds to the list of geometric transformation. For example:

$$h : \text{lines}, \text{cells} \xrightarrow{u} \text{lines},$$

with `h.pr.type = ''change the color of the 1st using the color of the 2nd''`, and $\gamma_h$ as above. In the state of these propagators we include some kind of link to the code that recognize this relation (e.g. the dictionary as above).

## 3. Fuzzy logic based neurotransmitters

Neurotransmitters are chemical messengers that the nervous system uses to transmit signals between neurons or from neurons to muscles. They regulate nearly all human functions, from mood to heart rate. In particular, dopamine implements a reward system, since it is essential for motivation, pleasure, learning, and motor control, see [14]. The fuzzy logic based indicators we implement in HiS-ARC can be thought of as a **very** rough implementation of dopamine (see e.g. [21, 22]): the more one of these indicators is near to 1, the more HiS-ARC labels the current hypotheses it is evaluating (see Sec. 9) as a promising one. The more one of these indicators is near to 0, the more HiS-ARC labels these hypotheses as a non-promising ones; under a certain level of fuzzy truth, HiS-ARC will (possibly temporarily) halt these hypotheses to consider a different set of assumptions. See also Sec. 10.

Up to now, the first idea for the general metacognition algorithm is to evaluate the fuzzy logic based dopamine of each part of the human inspired algorithms (entities, interactions, analogical mappings, invariance extraction, conditional mappings, etc.) The evaluation of each single choice is called *local dopamine*. HiS-ARC creates from these algorithms a series of 3-4 hypotheses. Using also synaptic plasticity, we start with an hypothesis and evaluate each local dopamine, then we add all the (possibly weak) local dopamine and insert this sum into a sigmoid function (in order to consider threshold and saturation effects). The output of this function is the *global level of dopamine* for this HiS-ARC hypothesis. In general, HiS-ARC could have more that one hypotheses and evaluate the global dopamine for all of them in order to decide what is the best hypothesis. If one started hypothesis has a too much low level of global dopamine, then HiS-ARC can stop it and start/add a new hypothesis in a tree algorithm. If the started hypothesis has a sufficiently good level of global dopamine, it automatically delete all the other hypotheses. Hypotheses that exchange a greater level of information (diversification; this corresponds to Contextual Constraints, see [5]) with the present situation are preferred. The winner of all these hypotheses should the best evaluation of the GEP (minimal costs and maximal diversification; this is Lateral Inhibition, see [5]). See 10.

3.1. **A fuzzy logic way to have the same rule in "all" the training puzzles.** An example going in the direction of a local fuzzy-logic-based dopamine indicator for an evaluation that "the same rule has to be applied in "all" training puzzles". In some examples, see e.g. `d4469b4b`, the same abstract entity cannot be found in all output training $\mathcal{S}_k$. In general, we have a fuzzy evaluation based on the number of training puzzles where we can apply the same rule. Clearly, when we have exactly the same rule applied in all the training puzzles, then this fuzzy value is maximum.

Some crisp/fuzzy costs can also be considered in a fuzzy based indicator (they reduce to zero the local dopamine level), such as:

1. The abstract descriptions of actions are different in the training puzzles.
2. The mappings $U$ and $R$ (e.g. obtained from analogical mappings) are not cognitive mappings (they are not CEF). See e.g. `beb8660c`.
3. The RLE or RLS property, i.e. $U \vdash R$ or $L \vdash R$, does not hold.

This corresponds to Prediction Error Minimization (see [5]).

## 4. Synaptic plasticity and object affordance

In general and intuitive terms, *synaptic plasticity* is the human brain's ability to reshape and constantly change itself in response to our experiences, thoughts, and environment, see [18]. The more a thought, action, or memory is repeated, the wider, smoother, and probable is for a human to use this cognitive path: In the brain, the connections (synapses) between neurons strengthen (plasticity). A related behavior seems to be *object affordance*, i.e. the phenomenon where the human brain is automatically pre-alerted or prepared for potential interactions upon seeing a well-known object, see [15]. We could say that humans have a greater probability to activate on an object if they have seen it several times.

We implement synaptic plasticity as a CAS because HiS-ARC has to decide among different competitive interpretations. It *could be* better to implement synaptic plasticity for *everything where we have potential multiple choices*: choice of entities or interactions, different choices to construct an analogical or cognitive mappings, even different ways to linearize a matrix (i.e. to transform it into a row) or one among the two possible orders in a linear pattern.

4.1. ***Synaptic plasticity* algorithm following the GEP.** We want to solve the problem "how to use a human-inspired algorithm in order to find what choice among several ones (e.g. geometric entities, interaction, actions, etc.) is better to consider first in the given training puzzle (IS), avoiding a brute-force algorithm". We will use the GEP and hence we are implementing a CAS:

1. At the initial time $t_{\text{st}}$ all the counting goods $\gamma_{i_1}, \dots, \gamma_{i_N} = 1$. The interpretation is: HiS-ARC already has a human-inspired core-knowledge which allows it to make a choice $i_k$ among $k = 1, \dots, N$ different choices (e.g. what entity occurs, or what interaction, or what action creates a given entity in an output training or even what order HiS-ARC has to use among the two in a linear pattern). However, initially at $t = t_{\text{st}}$ HiS-ARC has no experience about how frequently $i_k$ occurs in ARC-AGI 1, 2 problems, but it will adapt its behaviour in learning to solve these puzzles (this is plasticity, i.e. adaptation based on occurring frequencies of choices $i_k$, see [18]).
2. At generic step: By normalization, transform all the counting goods into a probability. At the initial time $t_{\text{st}}$, we clearly get a uniform probability.
3. HiS-ARC makes a first choice $i_k$ using this probability, e.g. it searches for an interacting entity of type $k$ among all the $N$ it is able to recognize.
4. HiS-ARC evaluates the local dopamine for this choice $i_k$, e.g. it searches for occurrences of this object in "all" (fuzzy) the training puzzles.
   In the language of IS, the fact that the entity $g$ is found in $\mathcal{I}_j$ corresponds to activate an interaction $i_k : g \xrightarrow{v} g$ as described in 2.1 in Sec. 2.1, i.e. setting $\text{ac}^v_{i_k}(t) = 1$ (see also below 6.). We must proceed similarly for all the other possible choices: interactions, actions, states, etc.
5. If HiS-ARC evaluates a sufficiently high value of local dopamine, e.g. if $i_k$ occurs in "all" the training IS, then it finds if some $i_j$ of lower abstraction level (see 2.1) occurs in the training puzzle and has the same level of local dopamine, until it founds the lower abstraction level (e.g. it could happen that the firstly extracted $i_k$ is `'closed lines'` whereas at the end the lower abstraction level $i_j$ is `'rectangles'`).
6. The found lowest abstraction level has to correspond to an interaction $i_j : g \xrightarrow{v} g$ as described in 2.1, i.e. we can say that $\text{ac}^v_{i_j}(t) = 1$. Increase its counting goods: $\gamma_{i_j}(t+1) = \gamma_{i_j}(t) + 1$ ($t$ is not actually implemented in the code and intuitively corresponds to "next step in the algorithm"). Drop $i_j$ from the list $i_1, \dots, i_N$ and restart from 3..
7. If HiS-ARC evaluates that the level of dopamine is under a fixed threshold, e.g. $i_k$ does not occurs in "all" training IS, then it searches for occurrences at a higher abstraction level.
8. The higher found abstraction level is one of the $i_j$. Increase its counting goods: $\gamma_{i_j}(t+1) = \gamma_{i_j}(t) + 1$. Drop $i_j$ from the list $i_1, \dots, i_N$ and restart from 3..
9. If no higher abstraction level occurs in all training puzzles, then drop $i_k$ from the list $i_1, \dots, i_N$ and restart from 3..
   In the language of IS and the GEP, this corresponds to have $Q_{\gamma_{i_k}(t+1)}(i_k) = 0$ in the diversification probability or

deactivation of of the propagator $\mathrm{pr}(i_k)$, $\mathrm{ac}_{i_k}^{\mathrm{pr}(i_k)}(t+1)=0$, and setting a maximal value of cost $C^h_{y(t+1)}(g)=1$, where $i_k : g \xrightarrow{v} g$ and the global state $y(t+1)$ contains the new state $\mathrm{ac}_{i_k}^{\mathrm{pr}(i_k)}(t+1)=0$).

10. With the new counting goods, restart from 2..

It is possible to prove that, at each step in this synaptic plasticity algorithm, the normalized probability is actually the maximum entropy probability on $i_1,\dots,i_N$ that "respects the past HiS-ARC experience", i.e. the counting goods experienced by HiS-ARC in the past. In fact this probability maximize the entropy among all probabilities $p_1,\dots,p_N$ such that $p_k(t) \geq \frac{\gamma_{i_k}(t)}{\sum_{j=1}^N \gamma_{i_j}(t)} =: P_k(t)$ for all the $i_k$ that HiS-ARC already found in the past, $k=1,\dots,M \leq N$. Normalization also corresponds to assign "the remaining probability" $p_j(t)=\frac{1-\sum_{k=1}^M P_k(t)}{N-M}$ to all not-founded objects $j=M+1,\dots,N$. Therefore, this synaptic plasticity algorithm (is very simple and natural, and) satisfies the GEP (i.e. it implement synaptic plasticity as a CAS).

However, we have to consider that this synaptic plasticity algorithm improves HiS-ARC behavior only with respect to the frequency a choice is used: the more frequently a choice has been used in the pasg, the most probable it will be that HiS-ARC will firstly make this choice. A different problem is *contextualisation*, i.e. understanding and interpreting data within the specific setting of a given environment. This is considered in the following section.

4.2. ***Contextualisation* and *Object affordance* algorithm.** For an intuitive first description of *object affordance*, see [15]. This algorithm can also be seen as a more efficient (with respect to a pure brute-force one) way to recognize what actions could be performed on the entities we have just found in the input training puzzles. We describe it for recognition of entities, but in principle it could be applied to any kind of HiS-ARC choice (see also Situated Conceptualization in Sec. 5 for a very similar algorithm):

1. Recognize occurrences of interacting entities in each training IS $\mathcal{I}_k$, $\mathcal{S}_k$ using the previous synaptic plasticity algorithm.
2. Select only interactions/relations and actions/transformations having these interacting entities as *patients* or *agents* (once again, this selection is an application of the GEP, setting to zero the probability to have interactions with other entities; indeed, it is a way to formalize the idea "consider only interaction and actions that exchange information with the entities HiS-ARC recognized in inputs). For this selection, we have to use the abstract interactions described in Sec. 2.2 and Sec. 2.3.
3. Using the counting goods of these interactions/actions and the related synaptic plasticity, recognize what relations and transformations occur in (fuzzy) "all" the training IS.
4. After the previous step 2., we have to consider other 2 or 3 possible compositions of abstract actions $b_1,\dots,b_n \xrightarrow{r} a_k$ ending up with patients which are agents of transformations $a_1,\dots,a_m \xrightarrow{s} p$ considered in 2..

Because of the nature of synaptic plasticity algorithm, we can state that using this object affordance algorithm HiS-ARC is automatically pre-alerted or prepared for potential interactions/actions upon seeing a well-known object, i.e. it has a greater probability to act on an object if it has seen it several times. Similarly, we have to implement object affordance of other type of choices, such as interactions, actions, states, analogies, etc.

4.2.1. *Retrospective study of synaptic plasticity.* We can also study the final results of synaptic plasticity after the solution of "all" the training examples: what entities are more probable, what interactions?

Some effect of synaptic plasticity that we can check a posteriori:

1. HiS-ARC should prefer "set of cells of a certain type" instead of "single cell"
2. Columns/rows/diagonals made of cells of the same colors.

## 5. Analogical mappings using Situated Conceptualization (SitCon)

In *Analogical Mapping*, interacting entities are not placed in analogy because they look physically alike, but because the interactions between components in the *Source* system are identical to the internal relationships in the *Target* system, see e.g. [3, 4]. We formalize and implement this type of mappings using *Situated Conceptualization* (called "method of the chair" in the UGM mini-course or workshop). According to Situated Conceptualization, the brain does not retrieve static definitions of objects, but rather prepares for action: an object is recognized by fusing its stable physical properties with the dynamic, interactive scenario in which it is placed. In other words, Situated Conceptualization is the human process where the brain recognizes an object using some properties of its state, but mainly using the CE interactions this object is able to perform as an agent or a patient. Said with a slogan: "A chair is what you can do with a chair + some state properties such as having a seat place where one can sit, and having a seat back". In our setting, we use the shorter name *SitCon* instead of "Situated Conceptualization". For example, between two rectangles of different form (e.g. height, width and colors) we can construct a map by matching L-forms around vertices and sides of the two rectangles, so that the "touching" geometric interactions are preserved: this

is a cognitive mapping (see Sec. 6). If we do not care about the preservation of the CE interactions or we are only preserving some properties (in a fuzzy way), we have an *analogical mapping*. For example, if we evaluate in a fuzzy way the L-forms of the aforementioned example with two rectangles, we have a possible analogical mapping.

Some general names:

**Horizontal analogical/cognitive mappings::** Are mappings of the type $\mathcal{I}_k \longrightarrow \mathcal{S}_k$.

**Vertical analogical/cognitive mappings::** Are mappings of the type $\mathcal{I}_k \longrightarrow \mathcal{I}_j$ or $\mathcal{S}_k \longrightarrow \mathcal{S}_j$ with $k \neq j$. Vertical analogical or cognitive mappings correspond to human's *global* vision of everything in the puzzle.

**Internal analogical/cognitive mappings::** Are mappings of the type $\mathcal{I}_k \longrightarrow \mathcal{I}_k$ or $\mathcal{S}_k \longrightarrow \mathcal{S}_k$.

It seems that analogical or cognitive mappings of the form $\mathcal{I}_k \longrightarrow \mathcal{S}_j$, with $k \neq j$, are not useful in ARC-AGI 1, 2 puzzles.

A first version of the algorithm used to construct analogical mappings is as follows (we describe how to construct horizontal analogical mappings $U_k : \mathcal{I}_k \longrightarrow S_k$. Similarly, we can proceed for the other type of mappings):

1. We imagine to have already recognized geometric entities, relations and transformations using the synaptic plasticity algorithm described above in Sec. 4.
2. For each entity $e \in \mathcal{I}_k$:
   (a) Consider the state $x_e$.
   (b) Consider all the interactions $a_1, \ldots, a_n \xrightarrow{r} e$ in $\mathcal{I}_k$ where $e$ is a patient.
   (c) Consider all the interactions $a_1, \ldots, e, \ldots, a_n \xrightarrow{r} p$ in $\mathcal{I}_k$ where $e$ is an agent.
   (d) Consider all the compositions of 2-3 transformations/actions taking $e$ as *input*, e.g. $T : \mathcal{I}_k \longrightarrow S_k$ and the related code `T(e)` yields no error.
3. For each entity $\bar{e} \in \mathcal{S}_k$:
   (a) Consider the state $x_{\bar{e}}$.
   (b) Consider all the interactions $b_1, \ldots, b_m \xrightarrow{\bar{r}} \bar{e}$ in $\mathcal{S}_k$ where $\bar{e}$ is a patient.
   (c) Consider all the interactions $b_1, \ldots, \bar{e}, \ldots, b_m \xrightarrow{\bar{r}} \bar{p}$ in $\mathcal{S}_k$ where $\bar{e}$ is an agent.
   (d) Consider all the composition of 2-3 transformations/actions having $\bar{e}$ as *output*.
4. Compare:
   (a) Each `e.states_description` with `bar_e.states_descrition`: crisp comparison
   (b) Each value `e.x[c]` with `bar_e.x[d]` for each pairs of `c`-th and `d`-th components of the states (they may have different dimensions): fuzzy comparison. In case we do not have a more meaningful way to fix a fuzzy logic based comparison, we can simply set a triangular fuzzy membership function `comp` centered at zero and evaluate `comp(e.x[c] - e.x[d])` if the states are numbers.
   (c) The `type` of each interaction (its propagator's attribute). We can think this type as an abstract description of the interaction.
   (d) Determine if the (composition of 2-3) action $T : \mathcal{I}_k \longrightarrow S_k$ satisfies $T(e) = \bar{e}$. If the same action $T$ is used for entities $e$ of different abstract types, then increase the global value of dopamine (depending on the number of different abstract types).
   (e) Apply the previous comparison to $(a_1, \ldots, a_n)$ and $(b_1, \ldots, b_m)$, or $r$ and $\bar{r}$, or $p$ and $\bar{p}$.
   (f) For each one of these comparisons criterion resulting in a high fuzzy value (local dopamine) we define a different analogical mapping by setting `U(k)[e]=bar_e` or `U(k)[a_1,...,a_n]=(b_1,...,b_m)` or `U(k)[a_1,...,e,...,a_n]=(b_1,...,bar_e,...,b_m)` and `U(k)[r]=bar_r`, etc. depending on the criterion. We always have to think that these mappings are precursors of cognitive mappings (i.e. CEF), and that the latter transform CE arrows in one IS into CE arrows of another IS.
   (g) The comparison rule used above has to be used in "all" the training puzzles.
5. Evaluate the local dopamine by adding all the previous fuzzy evaluations and inserting the final results into a sigmoid function. We can also consider this idea to summarize some of these criteria, but to create a fuzzy membership function depending on three values: this fuzzy sum, the number of criteria used in SitCom, of number of input training puzzles $\mathcal{I}_k$ and $\mathcal{S}_k$ where these criteria are used.
6. For all the other entities $e'$, set `U(k)[e']=∅`: these entities $e'$ are forgotten.

## 6. Cognitive mappings

In our setting, *cognitive mappings* are CEF, but see also [7]. Usually, they are defined by an analogical mapping, see Sec. 5, by checking that these maps actually preserve CE interactions from one IS (e.g. $\mathcal{I}_k$) to another IS (e.g. $\mathcal{S}_k$ for horizontal cognitive mappings). Therefore, their implementation follows exactly the algorithm described above in Sec. 5. The remaining key point is hence how to implement the verification that the mapping preserves CE interactions.

The mathematical definition of a CEF $U_k : \mathcal{I}_k \longrightarrow S_k$ (in a static IS) is: for all interactions $a_1, \ldots, a_n \xrightarrow{r} p$ in $\mathcal{I}_k$, we have $U_k(a_1, \ldots, a_n) \xrightarrow{U_k(r)} U_k(p)$ in $\mathcal{S}_k$. We recall that, in general, $U_k$ does not preserve arity: i.e. $U_k(a_1, \ldots, a_n) = (b_1, \ldots, b_m)$ but not necessarily $n = m$ (but if $n = 1$, then $m = 1$, which is implicit in the notations $U_k(r)$ and $U_k(p)$).
As we mentioned above, the CEF can be implemented as a tuple of dictionaries `U(k)[a_1,...,a_n]=(b_1,...,b_m)`, `U(k)[r]=bar_r`, `U(k)`[p]`=bar_p`. The mathematical definition of CEF can be implemented as follows:

```
# Cal_I[k] is the IS corresponding to the input training grid cal_I[k] loaded
# with load_json.py

# Put all the interactions of Cal_I[k] in a list
Cal_I_k_interactions = list(Cal_I[k].I.values())

# Put all the interactions of Cal_S[k] in a list
Cal_S_k_interactions = list(Cal_S[k].I.values())

# The following is the map U–taking interactions of Cal_I[k] to Cal_S[k]
U_interactions_map = {}

# for each interaction i in Cal_I[k]
for i in Cal_I_k_interactions:
        # initialize U_interactions_map at i
        U_interactions_map[i] = None

        # values of the CEF U
        agents_U = U(k)[i.agents]
        pr_U = U(k)[i.pr]
        pa_U = U(k)[i.pa]

        # for each interaction j in Cal_S[k],
        # check if the previous values of U form an interaction in Cal_S[k]
        for j in Cal_S_k_interactions:
                if j.agents == agents_U and j.pr == pr_U and j.pa == pa_U:
                        U_interactions_map[i] = j
                        break
        if U_interactions_map[i] == None:
                print(f"{U} is not a CEF because it doesn't preserves {i.id}")

print(f"{U} is a CEF!")
return U_interactions_map
```

*Remark* 3.

1. **Open problem:** Differently to what I stated during the UGM lectures, it seems more natural to consider that from the training pair $\mathcal{I}_k - \mathcal{S}_k$ we are constructing two cognitive mappings for RLE, one of which is a forgetful one (because some interacting entities are not considered at all in the solution). Moreover, in general, some actions to construct $\mathcal{S}_k$ from $\mathcal{I}_k$ need *more than one* interacting entity, such as: copy on $\mathcal{S}_k$ the rectangle in $\mathcal{I}_k$ but with a new color taken from the most numerous set of cells in $\mathcal{I}_k$. *Since we start from more than one interacting entity* (the rectangle and its interior cells), the natural learning is RLE.
2. On the contrary, in transforming the final empty $\mathcal{S}_N$ for the solution, we proceed stepwise starting from one interacting entity to the next one, and aiming to change its state. For example:
   (a) Change the dimension of the grid by following the rules used in the training and guessed by the Elementary Arithmetic Unit (EAU, see Sec. 8).
   (b) Change the color of this part of the grid by following the rules founded in the training puzzles.

(c) etc.

In this case, the natural learning process is therefore RLS because we start *from only one* interacting entity and we want to change its state.

3. It could be useful to see analogical and cognitive mappings as interacting entities of $\mathcal{A}bst$. In this way, we can e.g. talk of the state of a cognitive mapping, one of whose component could be its "abstract description". This state can be used to apply the fuzzy rule of considering the same abstract transformations in all the training pair $\mathcal{I}_k - \mathcal{S}_k$.
4. We can visualize a CEF $U(x) = y$ by changing the color of the border (1 pixel width) of each interacting entities: first $x$ and after a certain time $y$. We can also make disappear all the interacting entities that are forgotten ($U(x) = \emptyset$) by a cognitive mapping.

## 7. Invariance extraction and Conditional actions

*Invariance Extraction* is brain's ability to identify properties that remain equal in different settings, see e.g. [12]. After this identification, we can perfrom *Conditional Actions*, i.e. actions of the type "If the invariant property is $P$, then make action $A$".

In the examples we considered in [20], invariant properties are of the following type:

1. `868de0fa`: Squares have odd (or even) dimensions
2. `6d0160f0`:
   (a) The complex square has a yellow cell in its interior
   (b) The position $(x, y)$ of the yellow cell equals the position of the square in $\mathcal{S}_k$ taking the unit of measurement 3 cells = 1 unit
3. `760b3cac`: 1 + the $x$ coordinate of the vertical arm of the yellow L-form equals the abscissa used in the symmetry action
4. `995c5fa3`: "position number" in the linear patterns $\mathcal{I}_k$ and $\mathcal{S}_k$ are equal
5. `5117e062`: "having a light-blue cell touching other cells of the same color"
6. `a8c38be5`: the same type of L-form or T-form in $\mathcal{I}_k$ end up in the same position in $\mathcal{S}_k$
7. `855e0971`:
   (a) The linear pattern is vertical or horizontal
   (b) The abscissa of the black cells in $\mathcal{I}_k$ equals the abscissa of the black columns in $\mathcal{S}_k$
   (c) The ordinate of the black cells in $\mathcal{I}_k$ equals the ordinate of the black rows in $\mathcal{S}_k$
8. `cce03e0d`: the coordinate $(x, y)$ of each red cell in $\mathcal{I}_k$ equals the coordinate $(\bar{x}, \bar{y})$ of the copied square in $\mathcal{S}_k$ in the unit of measurement 3 cells = 1 unit in $\mathcal{S}_k$
9. `3e980e27`: "red cell" and "green cell" are the invariant properties for the conditional actions
10. `e8593010`: entities in $\mathcal{I}_k$ have the same number of cells
11. `d4469b4b`: Here we have conditional actions of type: "if color = $c$, then draw the form $X$", so the color is the invariant property.

Therefore, it seems that invariant properties are always of the form:

- Certain state properties of entities $e \in \mathcal{I}_k$ and $g \in \mathcal{I}_j$ are equal
- Certain state properties of entities $e \in \mathcal{I}_k$ and $\bar{e} \in \mathcal{S}_j$ are equal, where $\bar{e} = U_k(e)$ and $U_k$ is an analogical/cognitive mapping
- As above, but the equality is between elementary operations starting from numerical states of the considered entities (vertically or horizontally along a mapping).

Therefore, it seems that the following is a first attempt to find invariant properties and hence set the conditions in conditional actions:

- Search equal state properties (possibly between corresponding entities along an analogical mapping).
- The use of the EAU, see Sec. 8, in order to obtain the same result starting from the states of two entities.

## 8. Elementary Arithmetic Unit

The EAU is organized as a CAS with the GEP and RLS. Here is a rough algorithmic organization of this unit:

1. Count "everything" in the training puzzles: number of cells of a certain color (black included), number of cells, number of entities of the same abstract type, number of different colors, number of sub-entities in each given complex entity, number of analogical or cognitive mappings, number of entities having a certain property. Some of these numbers are better inserted in the state of the related entities once they are recognized.

2. Compute "everything" related to the entities HiS-ARC has recognized in the training puzzles: area of rectangles, length of sides, position of each entity, etc. Also some of these numbers are better inserted in the state of the related entities once they are recognized.
3. For each "problem" (e.g. HiS-ARC has to decide the final grid's dimensions or the numerical input value of an action), HiS-ARC creates a "correspondence table": for example, if the problem is "find the grid's dimension" in `feca6190`, then the table is $(n, d)$, where $n$ is a counting number from the previous two steps 1., 2. and $d$ is the final grid's dimension: in `feca6190`, we can e.g. have $n =$ number of red, or $\bar{n} =$ number of non-black and hence the tables are

| $n$ | $d$ |
|---|---|
| 0 | 10 |
| 1 | 5 |
| 0 | 15 |
| 0 | 15 |
| 0 | 5 |

| $\bar{n}$ | $d$ |
|---|---|
| 2 | 10 |
| 1 | 5 |
| 3 | 15 |
| 3 | 15 |
| 1 | 5 |

4. In general, the tables are of the form $((n_1, \dots, n_K), d)$, where $n_j$ are counting numbers (usually $K \leq 2$) and $d$ is the output number we want that the EAU computes. The general problem has to be thought of as "what kind of interaction between input numbers $(n_1, \dots, n_K)$ we have to consider in order to obtain $d$ in "all" training puzzles (i.e. in all the rows of the previous tables). We therefore have to find an interaction $n_1, \dots, n_K \xrightarrow{\alpha} d$, where $\alpha$ is among the core-knowledge of HiS-ARC elementary operations. Probably, for the EAU there is no need to consider "all" in a fuzzy sense, but only in a crisp sense.
5. Each pair in the previous table is a $\{0, 1\}$ cost: the chosen operation $\alpha$ works (cost $= 0$) or not (cost $= 1$), and the RL is RLS because we have to chose $\alpha$ in order to minimize all these costs for each row in the table (GEP).

Examples of elementary operations or properties we can consider in the core-knowledge of HiS-ARC:

1. The 4 elementary operations
2. The given number is even/odd
3. The linear pattern (or tiling) has a lower/greater number of elements
4. Entity $X$ precedes entity $Y$ in the linear pattern $P$
5. Set a unit of measurement in $\mathcal{I}_k$ or in $\mathcal{S}_k$, i.e. decide that $n$ cells corresponds to 1 unit. For example, this is the case if all entities in $\mathcal{I}_k$ or $\mathcal{S}_k$ have the same width and height $n$. If they only have the same width $n$, HiS-ARC can fix a unit of measurement only for abscissas.
6. Find the best linear mapping using the counting numbers appearing in $\mathcal{I}_k$ and $\mathcal{S}_k$: can we find a general numerical algorithm?
7. The middle entity in a linear pattern. The $k$-th entity in a linear pattern.
8. $x = y \bmod n$
9. Distance between "parallel" entities (e.g. rectangles one included in another one).

## 9. Hypotheses testing and predictive processing

*Hypotheses testing* and *predictive processing* is the process where the brain actively accumulates information from the environment in favor of or against the hypothesis currently "under review." Only when the evidence exceeds a certain critical threshold is the hypothesis accepted or definitively rejected, unlocking executive control to move on to the next candidate.

While our conscious mind (working memory) struggles to handle more than one hypothesis at a time when solving complex problems, the underlying brain is an extraordinary parallel machine. In fact, according to the Bayesian Brain theory, the nervous system never bets on a single explanation from the very beginning. Faced with an ambiguous stimulus, entire populations of neurons simultaneously encode different probability distributions (that is, multiple hypotheses in parallel). These hypotheses compete with each other at a sub-conscious level until one accumulates enough evidence to win, emerging into consciousness. In the brain's neural networks, different hypotheses undergo a process of lateral inhibition. Imagine three hypotheses active at the same time. Each neuronal representation tries to "turn off" the other two. Until the sensory data becomes clear, the brain maintains all three in a state of low-intensity parallel activation. We can hold in memory the names of 2 or 3 hypotheses simultaneously, but when we begin to actively test them by simulating their consequences, the cognitive load almost always forces us to focus internal attention on a single main hypothesis, performing a rapid serial shifting of attention between one and the other.

In examples [20], we have seen the following hypotheses testing and predictive processing:

1. a68b268e: HiS-ARC constructs an hypothesis for each possible composition of 4 actions (corresponding to 4 analogical mappings); at each step in this composition (e.g. after only the composition of two actions) it evaluates the costs and abandon this hypothesis if the costs are maximal (e.g. orange is not fully preserved); the composition working in all training is the winning hypothesis.
2. 91413438: HiS-ARC makes an hypothesis about what linearization method it has to use to repeatedly drawing an entity on the output grids.
3. 05269061: HiS-ARC uses analogical mappings and set several hypotheses for different colors of a pattern $y_N$ that have a good analogy with other patterns $y_k$ in the training puzzles. It tests each hypothesis until the "complete the pattern" algorithm works.
4. 40853293: the hypothesis is about the order of subsequent actions in drawing rows or columns. The correct order is "first rows, then columns" that works for the training puzzles. This order has to be applied for the final solution.
5. cce03e0d: HiS-ARC has to set several hypotheses because it is not able to get out from a certain step with a unique solution having a high value of local dopamine. For example, it has several different hypotheses for what entities it can recognize in the training puzzles. Some of these hypotheses yield a path having a higher value of global dopamine.
6. 4522001f: similar to the previous example.

In general, a first algorithmic approach to hypotheses testing could be as follows:

- At each step of the metacognition algorithm, see below, HiS-ARC can get as output a set made of several possibilities. Some of them already have a local value of dopamine or probability, e.g. from the value of analogy in analogical mappings or from synaptic plasticity.
- HiS-ARC then sets a certain set of hypotheses to test, clearly starting from those having the best values of dopamine (Bayesian Brain).
- At each subsequent step of the metacognition algorithm, HiS-ARC evaluates the global value of dopamine. If this is under a fixed threshold, then it abandons this set of hypotheses and considers a new one having a sufficiently high value of dopamine.
- We have to consider that the number of possible outputs does not seem huge, and hence that a standard laptop can consider this potentially exponential problem of testing all the subsets of hypotheses.
- On the other hand, we can try to organize the choice of subsequent sets of hypotheses by considering those that exchange more information with the best hypotheses that HiS-ARC evaluated so far, i.e. organizing the predictive processing as a CAS. In this case, it is important what are the (abstract) interactions HiS-ARC has to consider for each object we introduce in the model.
- We can also consider that the global dopamine evaluation of a given set of hypotheses is obtain by simply adding the local dopamine levels and hence inserting them into a sigmoid function. This evaluation for each subset of hypotheses could be sufficiently quick. We can therefore choose the shortest path in this (hyper)graph of interactions using Dijkstra's algorithm, see [10].
- We can also evaluate to execute in parallel different hypotheses on different processor's cores.

## 10. Metacognition algorithm

*Metacognition* is the capacity to monitor, evaluate, direct, and reflect upon one's own cognitive processes and states of knowledge, see [8]. In our point of view, this algorithm concerns "how we put together all the single cognitive processes and evaluate paths and hypotheses". In other words, it is the framework algorithm that HiS-ARC uses to coordinate all the cognitive processes we presented above: core-knowledge for entities and interactions recognition, abstraction, local and global levels of dopamine using fuzzy logic based indicators, synaptic plasticity, analogical and cognitive mappings, invariance extraction and conditional actions, hypotheses testing and predictive processing, RLE, RLS. Therefore, it seems that *metacognition* is a possible name for this summarizing general algorithm. **Open problem:** Other possible names are *cognitive control*, *fluid reasoning*, *active inference*.

### 10.1. **Metacognition algorithm.**

1. Recognize (all possible!) entities using synaptic plasticity algorithm and their occurrence in "all" training puzzles, Sec. 4.
2. Recognize interactions using synaptic plasticity algorithm and their occurrence in "all" training puzzles, Sec. 4.
3. Construct analogical mappings (horizontal, vertical, internal) using the algorithm explained in Sec. 5.
4. Find what input parameters are needed by these actions to run.
5. Use the EAU to find some of these input parameters.
6. Find invariant properties using the ideas presented in Sec. 7.

7. Create conditional action using the previous invariant properties. A conditional action for $\mathcal{I}_k$ of the type "If $P_{jk}, \ldots, P_{jk}$, then $A_j$" for $j = 1, \ldots, A$ actions has to be applied to all entities $e \in \mathcal{I}_k$ that share these properties.
8. Find where HIS-ARC can take these parameters considering state of recognized entities or the analog of these through the previous analogical mappings. These input parameters are frequently taken from entities sharing the invariant properties.
9. Set an hypothesis for each permutation of the actions $A_j$, $j = 1, \ldots, A$ (unless this order is the same in all the training puzzles). Delete the hypotheses that are not able to reproduce the input training puzzles. If there is no criteria to prefer one hypothesis with respect to the other ones, set an equal dopamine value for all of them.
10. Each analogical mapping has to define a cognitive mapping. Delete analogical mapping that do not preserve CE interactions.
11. Find possible missing entities or interactions in $\mathcal{I}_k$ starting from entities in $\mathcal{S}_k$ that do not have a counter-image along any cognitive mapping. Use RLS to find these missing entities or interactions. A first cost to consider in this RLS is that now some cognitive mapping works better, i.e. has a higher value of local dopamine. Restart from 1. or 2..
12. For each cognitive mapping, define another one in the opposite direction as potential right or left adjoint CEF. If they do not satisfy the RLE or RLS condition, they are deleted.
13. Each one of the previous step (cognitive process) define an interaction/link in a (hyper)graph where entities/nodes are simply "step $n$ in an algorithm considering all these interactions/link". Use Dijkstra's algorithm to define an order of best hypotheses about how to use these cognitive processes. Evaluate the global dopamine for each one of these hypotheses and use the algorithm presented in Sec. 3 and Sec. 9.
14. Find grid's dimension in $\mathcal{S}_N$using EAU
15. Construct an abstract characterization of each cognitive process considered under a certain hypothesis (path in the graph mentioned in 13.): using SitCon, identify entities, interactions, actions, etc. by their state or by interaction where they are patients or agents. Consider also type of interactions, type of actions.
16. Use RLS and the previous abstract characterization to start changing $\mathcal{S}_N$. Conditional actions can be thought of as a program of actions whose input parameters are found as we specified above. Each abstract characterization has an English verbal description.
17. Reconstruct in $\mathcal{I}_N$, $\mathcal{S}_N$ the same number of cognitive mappings we considered in the training puzzles. Check RLE and RLS in the final solution. At the end the final value of global dopamine has to increase by adding the final solution.
18. Delete unused entities, interactions, actions, etc. and construct a natural language explanation of the final solution by concatenating the verbal description of each action.

*Remark* 4.

1. Implement the possibility to enter into a "verbose mode" that explain all reasonings and attempts (e.g. different path of cognitive processes), up to the final correct one.

## References


[1] Buzsáki, G. *The Brain from Inside Out*. Oxford University Press, 2019.
[2] Fleming, Towards an integrative neuroscience of metacognition
[3] Gemini - Analogical_mapping_examples
[4] Gemini - Analogical_cognition_and_brain_mechanisms
[5] Gemini - Brain organization
[6] Gemini, Buzsaki's "Action".
[7] Gemini - Cognitive mappings
[8] Gemini, Cognitive processes.
[9] Gemini, Core-knowledge: Innate cognitive abilities.
[10] Gemini - Dijkstra for B-hypergraphs and applications
[11] Gemini - Hypothesis testing
[12] Gemini - Invariance extraction
[13] Gemini, Neuroscience unconscious processes
[14] Gemini, Neurotransmitters and fuzzy logic
[15] Gemini, Object Affordance & Motor Priming
[16] Gemini, SitCon Situated Conceptualization
[17] Gemini, SitCon and Yoneda lemma
[18] Gemini, Synaptic plasticity.
[19] Hassabis, D., Kumaran, D., Summerfield, C., Botvinick, M. (2017). Neuroscience-Inspired Artificial Intelligence. Neuron, 95(2), 245-258.
[20] random quickly tested examples.lyx

[21] Lövheim, H., A new three-dimensional model for emotions and monoamine neurotransmitters. Medical Hypotheses, Volume 78, Issue 2, 2012, Pages 341-348, https://doi.org/10.1016/j.mehy.2011.11.016

[22] Zadeh, L.A., Toward a theory of fuzzy information granulation and its centrality in human reasoning and fuzzy logic. Fuzzy Sets and Systems, Volume 90, Issue 2, 1997, Pages 111-127, https://doi.org/10.1016/S0165-0114(97)00077-8.